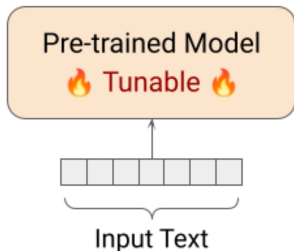


Deep Learning for NLP II

Fine-Tuning Large Language Models

Model Tuning (a.k.a. “Fine-Tuning”)



Learning goals

- BERT-style vs. SFT vs. prompting
- Parameter-efficient fine-tuning
- Adapter architectures
- (Q)LoRA for efficient adaptation
- Contrastive fine-tuning for embeddings

RECALL: PRE-TRAIN / FINE-TUNE PARADIGM

- **Pre-training:** Train a large model on massive unlabeled data
 - Encoder-only (BERT): Masked Language Modeling (MLM)
 - Decoder-only (GPT): Causal Language Modeling (CLM)
 - Encoder-Decoder (T5): span corruption / seq2seq objectives
- **Fine-tuning:** Adapt the pre-trained model to a downstream task
 - The pre-trained weights serve as an initialization
 - Much cheaper than training from scratch
 - Even modest amounts of labeled data suffice
- **Question:** But *how* do we adapt/fine-tune? There is more than one answer!

A TAXONOMY OF FINE-TUNING STRATEGIES

Three major paradigms:

❶ BERT-style Fine-Tuning

Update *all* model parameters; add a small task-specific head

❷ Supervised Fine-Tuning (SFT) for Generative LLMs

Fine-tune a decoder on (instruction, response) pairs; no new head needed

❸ Prompt-based Learning

Reformulate the task as a language modeling problem; minimal or *no* gradient updates

These strategies differ in *what* is updated, *how* the task is represented, and *how much* labeled data they need.

BERT-STYLE FINE-TUNING (1)

Setup:

- Pre-trained encoder (BERT, RoBERTa, ...) with weights θ_0
- Add a task-specific *classification head* on top of [CLS] token
- Update **all** parameters θ jointly on labeled data

Objective (classification):

$$\mathcal{L} = - \sum_{(x,y) \in \mathcal{D}} \log p_{\theta}(y \mid x)$$

Key properties:

- One separate model per task (different fine-tuned copies)
- Strong performance on many NLU benchmarks,
e.g., ▶ GLUE (Wang et al., 2019a) ▶ SuperGLUE (Wang et al., 2019b)
- Requires labeled data; **not** a generative approach

BERT-STYLE FINE-TUNING (2)

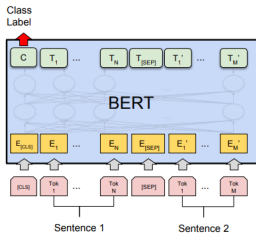
Supported task types:

- **Sequence classification:** Pool [CLS] → linear layer → softmax
Examples: sentiment analysis, NLI, document classification
- **Token classification:** Use each token's representation → per-token linear; Examples: NER, POS tagging, chunking
- **Span extraction / QA:** Predict start & end positions
Example: SQuAD reading comprehension

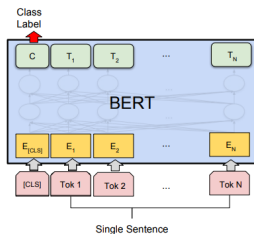
Practical hyperparameters: ▶ Devlin et al., 2019

- Adam lr: 2e-5, 3e-5, 5e-5; Batch size: 16, 32; Epochs: 2–4

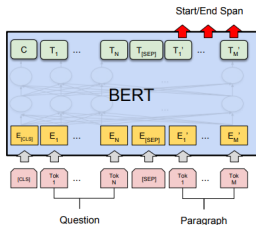
FINE-TUNING BERT



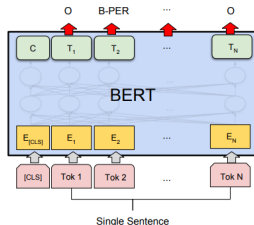
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

► Source: Devlin et al., 2019

SUPERVISED FINE-TUNING (SFT) FOR GENERATIVE LLMS

Generative LLMs (GPT-2/3/4, LLaMA, Mistral, ...):

- **Architecture:** Decoder-only Transformer
- **Pre-training:** Autoregressive LM $\rightarrow$ predict next token
- **No separate classification head** – output is free-form text

SFT: Fine-tune on curated (instruction, response) pairs

$$\mathcal{L}_{\text{SFT}} = - \sum_t \log p_{\theta}(y_t \mid x, y_{<t})$$

- Teach the model to *follow instructions*
- Examples: summarize, translate, answer questions, write code
- Foundation for RLHF / DPO alignment pipelines

PROMPT-BASED LEARNING

Core idea: Reformulate the task so it *looks like* the pre-training objective

- **Zero-shot prompting:**

“Classify the sentiment of: [text]. Answer positive or negative.”

→ No gradient update; relies entirely on pre-trained knowledge

- **Few-shot (in-context) learning:**

Prepend a few labeled examples to the prompt; still no training

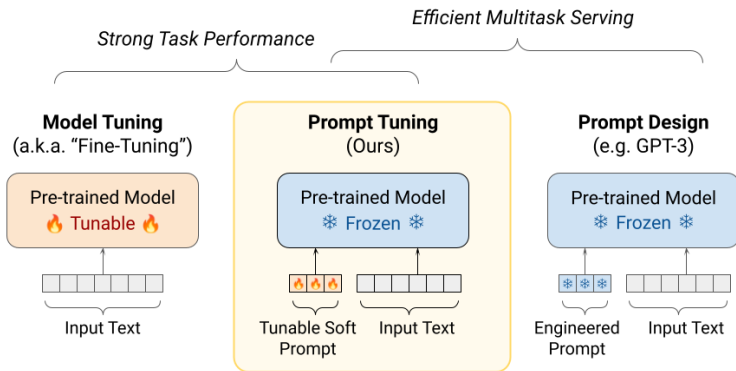
- **Prompt tuning / prefix tuning:**

Learn a small set of soft (continuous) prompt tokens; freeze the

LLM ▶ Lester et al., 2021 ▶ Li & Liang, 2021

Key benefit: One shared model for all tasks; no task-specific copies

PROMPT-TUNING



► Source: Lester et al., 2021

COMPARISON: FT VS. SFT VS. PROMPT-BASED

Property	BERT Fine-Tune	SFT (Gen. LLM)	Prompt-Based
Architecture	Encoder	Decoder	Both
Updated params	All $ \theta $	All $ \theta $	Few / None
Task representation	Classification head	Free text	In-context / soft prompt
Labeled data needed	Many	Moderate	Few / None
One model per task?	Yes	Often	No
Generative?	No	Yes	Yes

Question: Why does updating *all* parameters become a problem at scale?

PROBLEM: FULL FINE-TUNING AT SCALE

- GPT-3: $|\theta| = 175\text{B}$ parameters
- Storing one fine-tuned copy requires ≈ 350 GB (fp16)
- Deploying k tasks $\Rightarrow k \times 350$ GB storage
- Training requires optimizer states:
 - Adam requires $\sim 3 \times$ model size in memory

Consequence: Full fine-tuning of very large models is

- Prohibitively *expensive* (compute, memory, storage)
- Prone to *catastrophic forgetting* of pre-trained knowledge
- Inefficient when deployed *across many tasks*

→ **Parameter-Efficient Fine-Tuning (PEFT)** methods address this!

PEFT: OVERVIEW

Goal: Adapt a large pre-trained model to downstream tasks while updating only a *small fraction* of parameters

Main families:

- **Adapter layers** – insert small bottleneck modules *inside* each Transformer block ► Houlsby et al., 2019
- **Prefix / Prompt Tuning** – prepend learnable tokens to the sequence ► Lester et al., 2021
- **Low-rank decomposition (LoRA)** – approximate weight updates with low-rank matrices ► Hu et al., 2022
- **Selective / sparse updates** – update only a subset of existing parameters (e.g., BitFit)

ADAPTER LAYERS: CORE IDEA

Proposed by Houlsby et al. (2019):

- Insert small *adapter modules* after each sub-layer of the Transformer
- The pre-trained weights are **frozen**; only adapters are trained
- Each adapter is a *bottleneck* network:

Adapter architecture:

$$\text{Adapter}(h) = h + W_{\text{up}} \cdot \sigma(W_{\text{down}} \cdot h)$$

- $h \in \mathbb{R}^d$: hidden state from Transformer layer
- $W_{\text{down}} \in \mathbb{R}^{m \times d}$, $W_{\text{up}} \in \mathbb{R}^{d \times m}$, with $m \ll d$ (bottleneck)
- Residual connection ensures stability at initialization ($W_{\text{up}} W_{\text{down}} \approx 0$)

ADAPTER LAYERS: PLACEMENT

Placement in the Transformer block:

- After each **MHA** sub-layer
- After each **FF** sub-layer
- Each followed by layer norm
- Two modules per Transformer layer

BERT-base: ($d = 768$, $L = 12$):

- $m = 64$: $\approx 0.5\text{M}$ trainable params
- $m = 256$: $\approx 3.6\text{M}$ trainable params
- vs. $\approx 110\text{M}$ total (BERT-base)

Layer Norm
Feed-Forward
Adapter
Layer Norm
Multi-Head Attn
Adapter

ADAPTER LAYERS: KEY PROPERTIES

Advantages:

- **Parameter efficiency:** $< 1\%$ additional parameters per task
- **Shared backbone:** One pre-trained model; swap only small adapters
- **No catastrophic forgetting:** Frozen backbone preserves pre-trained knowledge
- **Modularity:** Adapters for different tasks / languages can be combined

Limitation:

- Adapters are processed *sequentially* $\Rightarrow$ added inference latency
- In online inference (batch size = 1), latency increase can be $> 20\%$ ► Hu et al., 2022
- Harder to exploit hardware parallelism (vs. frozen weights merged at inference time)

ADAPTERHUB

Problem: Sharing and re-using trained adapters is not straightforward

► Pfeiffer et al. (2020) propose **AdapterHub**:

- A *repository* for pre-trained adapters (analogous to HuggingFace)
- Dynamic “stitching-in” of adapters into any compatible backbone
- Built on top of HuggingFace `transformers`; minimal code changes required
- Supports BERT, RoBERTa, XLM-R and more

Workflow:

- ➊ Download pre-trained backbone
- ➋ Load adapter weights from `AdapterHub.ml`
- ➌ Train only the adapter; share back to the hub

ADAPTERFUSION

Motivation: Can we *combine* adapters trained on different tasks without retraining?

Sequential fine-tuning and **multi-task learning** both suffer from:

- Catastrophic forgetting (sequential)
- Dataset balancing difficulties (multi-task)

AdapterFusion ► Pfeiffer et al. (2021) propose a *two-stage* learning algorithm:

- 1 **Knowledge extraction:** Train N task-specific adapters independently (frozen backbone)
- 2 **Knowledge composition:** Learn a *fusion layer* that attends over the adapters

ADAPTERFUSION: THE FUSION LAYER

Fusion mechanism (attention over adapter outputs):

At each adapter position, given hidden state h and adapter outputs $Z = [z_1, \dots, z_N]$:

$$\text{AdapterFusion}(h, Z) = \text{softmax}\left(\frac{(hW_Q)(ZW_K)^T}{\sqrt{d}}\right) (ZW_V)$$

- W_Q, W_K, W_V : small learnable projection matrices
- The backbone and individual adapters remain **frozen**
- Only the fusion layer parameters are updated in stage 2

Result: Non-destructive task composition – evaluated on 16 diverse NLU tasks, outperforms full fine-tuning and multi-task learning

LORA: MOTIVATION

Key observation ► Aghajanyan et al., 2020:

- Pre-trained language models have a low *intrinsic dimension*
- They can learn effectively even when projected into a smaller subspace

LoRA hypothesis ► Hu et al., ICLR 2022:

- The *weight updates* ΔW during adaptation also have low intrinsic rank
- We can represent $\Delta W \approx BA$ where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$,
 $r \ll \min(d, k)$

Consequence: Instead of updating the full weight matrix, train only the low-rank decomposition B and A – much fewer parameters!

LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Modified forward pass:

For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$:

$$h = W_0x + \Delta Wx = W_0x + BAx$$

- W_0 is **frozen** – no gradient flows through it
- $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ are **trainable**
- Rank $r \ll \min(d, k)$ (e.g., $r = 4$ or $r = 8$)

Initialization:

- $A \sim \mathcal{N}(0, \sigma^2)$ (random Gaussian)
- $B = 0 \Rightarrow \Delta W = BA = 0$ at the start of training
- Scaled by $\frac{\alpha}{r}$ (where α is a fixed constant)

LORA: PARAMETER SAVINGS

Example: GPT-3 175B, applying LoRA to W_q and W_v ($d = 12.288$)

- Full fine-tuning: $|\Delta\theta| = |\theta_0| = 175\text{B}$ parameters
- LoRA with $r = 4$: $|\Theta| = 2 \times 2 \times d \times r = 4 \times 12288 \times 4 \approx 200\text{k}$ per layer

Reduction: $\approx 10,000\times$ fewer trainable parameters!

Memory savings:

- No optimizer states needed for W_0 (frozen)
- Adam: $3\times$ less GPU memory vs. full fine-tuning
- No gradient computation through W_0

LORA: NO INFERENCE LATENCY

Key advantage over adapters:

- After training, we can *merge* the LoRA weights back into the backbone:

$$W = W_0 + BA$$

- The merged weight W has the *same shape* as W_0
- **Zero additional inference latency** – the merged model runs identically to the original

Task switching:

- Recover W_0 by subtracting: $W_0 = W - BA$
- Load a different $B'A'$ for a different task
- Very low memory overhead for switching

LORA: WHERE TO APPLY?

Transformer weight matrices:

- Self-attention: W_q, W_k, W_v, W_o
- Feed-forward: W_1, W_2

Empirical findings (Hu et al., 2022):

- Adapting W_q and W_v alone gives strong results
- Spreading rank across more weight matrices (lower r each) outperforms concentrating rank in one matrix
- Rank $r = 4$ or $r = 8$ typically sufficient; increasing beyond $r = 64$ gives diminishing returns

LoRA is orthogonal to many methods – can be combined with prefix tuning, quantization, etc.

LoRA: EXPERIMENTAL RESULTS

RoBERTa, DeBERTa (NLU):

- LoRA matches or exceeds full fine-tuning on GLUE benchmarks
- Only $\approx 0.3\%$ of parameters updated (vs. 100% for full fine-tuning)

GPT-2 (NLG – E2E, WebNLG, DART):

- LoRA outperforms adapter-based methods and prefix tuning
- Training throughput $\approx 25\%$ higher than full fine-tuning

GPT-3 175B:

- LoRA with $r = 4$ matches full fine-tuning on WikiSQL and SAMSum
- Full fine-tuning of GPT-3 requires $\approx 1.2\text{TB}$ of GPU memory;
LoRA: $\approx 350\text{ GB}$

QLORA: EFFICIENT FINE-TUNING OF QUANTIZED LLMS

Challenge: Even with LoRA, fine-tuning a 65B parameter model requires > 780 GB GPU memory (fp16)

QLoRA ► Dettmers et al., 2023 combines:

- ❶ **4-bit quantization** of the pre-trained model (backbone frozen at 4-bit)
- ❷ **LoRA adapters** trained in bf16/fp16 on top of the quantized backbone
- ❸ Gradients are backpropagated *through* the quantized backbone into the LoRA parameters

Result: Fine-tune a 65B model on a *single 48 GB GPU*

QLORA: KEY INNOVATIONS

(a) 4-bit NormalFloat (NF4):

- New data type, information-theoretically optimal for normally distributed weights
- Outperforms standard 4-bit integers and 4-bit floats empirically

(b) Double Quantization:

- Quantize the quantization constants themselves (second level of compression)
- Saves ≈ 0.37 bits per parameter (≈ 3 GB for a 65B model)

(c) Paged Optimizers:

- Use NVIDIA unified memory to handle gradient checkpointing spikes
- Avoids out-of-memory errors when processing long sequences

QLORA: RESULTS – THE GUANACO MODELS

Guanaco family: 65B, 33B, 13B, 7B parameter models fine-tuned with QLoRA on OASST1 (9k samples)

Model	GPU mem.	Elo (Vicuna bench.)
GPT-4	–	1348
Guanaco 65B	41 GB	1022
Guanaco 33B	21 GB	992
ChatGPT	–	966
Guanaco 13B	10 GB	916

- Guanaco 65B reaches **99.3%** of ChatGPT performance (Vicuna benchmark)
- Trained in **24 hours** on a single GPU
- Data quality $\gg$ data size: 9k high-quality samples beat 450k FLAN samples

LORA VS. QLoRA: SUMMARY

Property	LoRA	QLoRA
Backbone precision	fp16/bf16	4-bit NF4
LoRA adapter precision	fp16/bf16	fp16/bf16
Memory saving vs. full FT	$\approx 3\times$	$\approx 16\times$
Inference latency	None (merge)	None (dequant + merge)
Performance vs. full FT	On-par	On-par
65B model GPU req.	~ 350 GB	< 48 GB

Bottom line: QLoRA makes fine-tuning frontier-scale models accessible on consumer hardware

FROM CLASSIFICATION TO EMBEDDINGS

Recap: BERT for classification

- Fine-tune with a cross-entropy classification head
- Optimizes *correct label prediction*, not embedding geometry
- The resulting embeddings are not necessarily good for retrieval / similarity

Other use cases requiring embeddings:

- Semantic textual similarity (STS)
- Dense retrieval / RAG (query $\leftrightarrow$ document matching)
- Clustering, deduplication, semantic search

Question: How do we fine-tune LLMs to produce *useful* sentence / document embeddings?

CONTRASTIVE LEARNING: CORE IDEA

Goal: Shape the embedding space so that:

- **Similar** sentences $\rightarrow$ **close** in embedding space
- **Dissimilar** sentences $\rightarrow$ **far apart** in embedding space

Triplet formulation:

- **Anchor** a : the query / reference sentence
- **Positive** p : a semantically similar sentence
- **Negative** n : a semantically dissimilar sentence

Triplet loss:

$$\mathcal{L}_{\text{triplet}} = \max(0, d(a, p) - d(a, n) + \epsilon)$$

where $d(\cdot, \cdot)$ is a distance metric (e.g., $1 - \cos(\cdot, \cdot)$) and $\epsilon > 0$ is a margin

SENTENCE-BERT (SBERT)

SBERT ► Reimers & Gurevych (2019):

- Fine-tune BERT with *siamese* and *triplet* network architectures
- **Siamese**: Two identical BERT encoders sharing weights; compare pooled representations
- **Objective**: Mean-pooled [CLS] output; trained on NLI and STS data

Training objectives used in SBERT:

- 1 **Classification (NLI)**: Concatenate $[u; v; |u - v|]$; softmax over entailment/neutral/contradiction
- 2 **Regression (STS)**: Cosine similarity of u and v ; MSE loss against gold similarity score
- 3 **Triplet objective**: Directly minimize triplet loss

SIMCSE: SIMPLE CONTRASTIVE LEARNING OF SENTENCE EMBEDDINGS

SimCSE ► Gao et al. (2021) uses *dropout* as minimal data augmentation

Unsupervised SimCSE:

- Pass the same sentence through the encoder *twice* with different dropout masks
- The two resulting embeddings form a **positive pair**
- All other sentences in the batch serve as **in-batch negatives**

In-batch negative loss:

$$\mathcal{L} = -\log \frac{e^{\text{sim}(h_i, h_i^+)/\tau}}{\sum_{j=1}^N e^{\text{sim}(h_i, h_j^+)/\tau}}$$

where τ is temperature and h_i^+ is the augmented positive for anchor h_i

SIMCSE: SUPERVISED AND HARD NEGATIVES

Supervised SimCSE:

- Use NLI data: premise as anchor, *entailment* as positive, *contradiction* as hard negative
- Objective includes hard negatives h_i^- :

$$\mathcal{L} = -\log \frac{e^{\text{sim}(h_i, h_i^+)/\tau}}{\sum_{j=1}^N \left(e^{\text{sim}(h_i, h_j^+)/\tau} + e^{\text{sim}(h_i, h_j^-)/\tau} \right)}$$

Why hard negatives matter:

- Random negatives are easy – the model learns little from them
- Hard negatives are superficially similar but semantically different
- Forcing the model to separate them sharpens the embedding space

CONTRASTIVE FINE-TUNING: DENSE RETRIEVAL

Dense Passage Retrieval (DPR) ► Karpukhin et al. (2020):

- Two separate BERT encoders: one for *queries*, one for *passages*
- Fine-tuned with in-batch negatives + hard negatives from BM25
- Positive pairs: (question, gold answer paragraph) from QA datasets

Modern recipe for embedding model fine-tuning:

- ➊ Start from a strong pre-trained LLM or BERT variant
- ➋ Mine (hard) negative pairs – e.g., using BM25 or a teacher model
- ➌ Train with contrastive loss + in-batch negatives
- ➍ Scale up: large batches, multi-GPU, GradCache for memory efficiency

MODERN EMBEDDING MODELS: E5, GTE, BGE

Commonalities of recent embedding models:

- Large LLM backbone (e.g., LLaMA, Mistral, or Qwen)
- Multi-stage training:
Weak Supervision → Supervised contrastive fine-tuning
- Massive mined datasets *with hard negatives*

Examples:

- **E5** ▶ Wang et al., 2022: Contrastive pre-training on CCPairs then SFT
- **BGE** ▶ Xiao et al., 2023: RetroMAE + contrastive SFT;
instruction-enhanced
- **GTE** ▶ Li et al., 2023: Multi-stage contrastive training pipeline

Key insight: Scale of contrastive data > scale of backbone parameters
for retrieval tasks

CONTRASTIVE FINE-TUNING: ANISOTROPY PROBLEM

Problem: representation degeneration ► Gao et al. (2019)

- BERT sentence embeddings (without contrastive training) cluster in a narrow cone
- High cosine similarity between semantically unrelated sentences
- The embedding space is *anisotropic*: most variance in one direction

Contrastive training fixes this:

- Pushes embeddings apart $\Rightarrow$ more uniform distribution on the hypersphere
- SimCSE measures this via *alignment* (positive pairs close) and *uniformity* (all points spread)

$$\mathcal{L}_{\text{uniform}} = \log \mathbb{E}_{x,y \sim p_{\text{data}}} \left[e^{-2\|f(x)-f(y)\|^2} \right]$$

CONTRASTIVE FINE-TUNING: PRACTICAL CONSIDERATIONS

Negative mining strategies:

- **Random in-batch negatives:** Simple, effective at large batch sizes
- **BM25 hard negatives:** Lexically similar but semantically different documents
- **Cross-encoder reranking:** Use a stronger model to score and mine hard negatives
- **Self-mined negatives:** Use the embedding model itself to find near-misses

Pooling strategies:

- **CLS pooling:** Use the [CLS] token representation
- **Mean pooling:** Average all token representations (often better)
- **Last token:** Common for decoder-only LLMs (EOS token)

SUMMARY: OVERVIEW ON STRATEGIES

Method	% Updated	Inf. latency	Multi-task	Use case
BERT FT	100%	None	No	NLU classification
SFT	100%	None	Limited	Instruction following
Prompt Tuning	< 0.1%	Minimal	Yes	Few/zero-shot
Adapters	1–3%	Added	Yes	BERT-era NLU
LoRA	0.1–1%	None	Yes	Gen. LLMs
QLoRA	0.1–1%	Minimal	Yes	Large Gen. LLMs
Contrastive FT	100% / partial	None	No	Embeddings, retrieval

SUMMARY: KEY TAKE-AWAYS

- **Three paradigms:** BERT-style FT (discriminative), SFT (generative), Prompt-based (minimal update)
- **Adapters:** Bottleneck modules inside the Transformer; frozen backbone; modular and composable (AdapterHub, AdapterFusion)
- **LoRA:** Represent weight updates as low-rank matrices BA ; no inference latency; $10,000\times$ fewer trainable params for GPT-3
- **QLoRA:** 4-bit backbone + LoRA adapters; 65B model on a single 48 GB GPU; data quality $\gg$ data quantity
- **Contrastive FT:** Shape the embedding space via positive/negative pairs; essential for retrieval, similarity search, and RAG pipelines